

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	ANUSHYA DEVI V	Reg.No.:	192424013
Department:		Date:	

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I ANUSHYA DEVI V (192424013) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
ANUSHYA DEVI V

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

ANSWER:

1. Debugging Signed Integer Addition Using 2's Complement

Step 1: Convert the Numbers to 8-bit Binary

- $+120 = 01111000$
- $+50 = 00110010$

Step 2: Perform Binary Addition

```
01111000 (+120)
+00110010 (+50)
-----
10101010
```

Step 3: Interpret the Result

- Binary result: **10101010**
- In 8-bit 2's complement, $10101010 = -86$
- Expected result: **+170**
- Actual result: **-86**

The result is incorrect because the 8-bit signed range is:

- Minimum = **-128**
- Maximum = **+127**

Since $170 > 127$, it cannot be represented in 8 bits.

Overflow Analysis

Overflow has occurred.

Overflow detection rule (2's complement):

- If two positive numbers are added and the result becomes negative, overflow has occurred.
- If two negative numbers are added and the result becomes positive, overflow has occurred.

Here:

- $+120 \rightarrow$ Positive
- $+50 \rightarrow$ Positive
- Result $\rightarrow$ Negative (10101010)

Overflow detected.

Another hardware rule:

- Overflow occurs when the carry into the sign bit $\neq$ carry out of the sign bit.

Cause of the Problem

- The program stores the result in an 8-bit signed integer. Since 170 exceeds the maximum value (127), the result wraps around to -86, producing an incorrect answer.

Proposed Solution

1. Check for overflow after every signed addition.
2. If overflow occurs:
 - Display an "Overflow Error" message, or
 - Use a larger data type (e.g., 16-bit or 32-bit integer) to store the result.

Final Answer

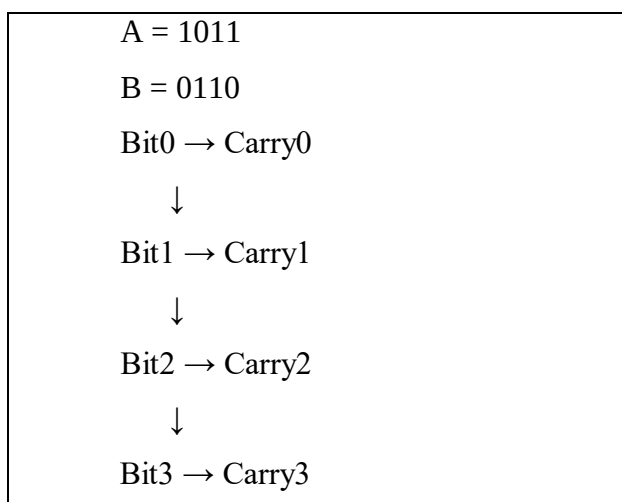
- Operands: +120 (01111000) and +50 (00110010)
- Binary Sum: 10101010
- Expected Decimal Result: +170
- Stored Result (8-bit): -86
- Overflow: Yes
- Solution: Detect overflow using sign-bit/carry-bit checks and use a larger integer size or report an overflow error.

2. Debugging Ripple Carry Adder (RCA) Performance Bottleneck

Step 1: Problem Analysis

A Ripple Carry Adder (RCA) adds binary numbers one bit at a time. Each full adder must wait for the carry from the previous stage before producing its output.

Example (4-bit addition):



The carry ripples from the Least Significant Bit (LSB) to the Most Significant Bit (MSB), causing delay.

Step 2: Performance Bottleneck

- Each bit must wait for the previous carry.
- Total delay increases with the number of bits.

- In real-time systems, this leads to slow arithmetic operations.

For an n-bit Ripple Carry Adder:

Total Delay $\approx n \times (\text{Full Adder Delay})$

Example:

- 8-bit RCA $\rightarrow$ 8 carry propagation delays
- 16-bit RCA $\rightarrow$ 16 delays
- 32-bit RCA $\rightarrow$ 32 delays

Step 3: Optimized Solution – Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder (CLA) computes carry signals simultaneously instead of waiting for each previous carry.

It uses:

- **Generate (G):** $G = A \times B$
- **Propagate (P):** $P = A \oplus B$

Carry equations:

$C1 = G0 + P0C0$ $C2 = G1 + P1G0 + P1P0C0$ $C3 = G2 + P2G1 + P2P1G0 + P2P1P0C0$

Since all carries are calculated in parallel, the sum is produced much faster.

Step 4: How CLA Reduces Delay

Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
Carry propagates bit by bit	Carries computed in parallel
High delay	Very low delay
Slow for large bit sizes	Fast for large bit sizes
Delay $\propto$ Number of bits	Much smaller delay

Final Answer

- **Problem:** Carry propagation through each bit causes significant delay in the Ripple Carry Adder.
- **Cause:** Each full adder waits for the previous carry before computing the next sum.
- **Optimized Solution:** Replace the Ripple Carry Adder with a Carry Look-Ahead Adder (CLA).
- **Benefit:** CLA calculates all carry bits in parallel using generate and propagate logic, greatly reducing delay and improving performance in real-time systems.

3. Debugging Booth's Multiplication Algorithm

Step 1: Problem Analysis

Booth's algorithm is used for signed binary multiplication (2's complement). Incorrect results for negative operands usually occur due to errors in:

- Bit-pair checking (Q_0 and Q_{-1})
- Arithmetic right shift
- Sign extension

Step 2: Booth's Algorithm Steps

Example:

- Multiplicand (M) = -5 = 1011 (4-bit)
- Multiplier (Q) = 3 = 0011

Initialize:

A = 0000
Q = 0011
Q-1 = 0
M = 1011

For each bit:

1. Check (Q_0, Q_{-1})

$Q_0 Q_{-1}$	Operation
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

2. Perform Arithmetic Right Shift on (A, Q, Q_{-1}).
3. Repeat for all bits.

Step 3: Possible Sources of Error

1. Incorrect Bit-Pair Checking

- Wrong interpretation of Q_0 and Q_{-1} .
- Incorrect add/subtract operation.

Correction: Follow the Booth decision table exactly.

2. Logical Shift Instead of Arithmetic Shift

- A logical right shift inserts 0 into the MSB.
- This changes the sign of negative numbers.

Correction: Use Arithmetic Right Shift, which copies the sign bit (MSB).

3. Incorrect Sign Extension

- Negative numbers must retain their sign during shifts.
- Failure to extend the sign produces incorrect results.

Correction: Replicate the MSB during every arithmetic shift.

4. Wrong Initialization of Q_{-1}

- Q_{-1} must always start with 0.

Correction: Initialize $Q_{-1} = 0$ before execution.

5. Incorrect Number of Iterations

- The algorithm must execute exactly n times for an n -bit multiplier.

Correction: Perform one iteration for each multiplier bit.

Step 4: Correct Execution

- Correct bit-pair checking (Q_0, Q_{-1})
- Proper add/subtract operation
- Arithmetic right shift with sign extension
- Correct initialization of Q_{-1}
- Execute exactly n iterations

Final Answer

- Problem: Incorrect results for negative operands during Booth's multiplication.
- Possible Causes: Incorrect bit-pair checking, logical right shift instead of arithmetic shift, improper sign extension, incorrect Q_{-1} initialization, or wrong iteration count.
- Corrections: Use the correct Booth decision table, perform arithmetic right shifts with sign extension, initialize $Q_{-1} = 0$, and execute exactly n iterations.
- Result: These corrections ensure accurate signed multiplication for both positive and negative operands.

4. Debugging Restoring Division Algorithm

Step 1: Problem Analysis

The Restoring Division Algorithm performs binary division by repeatedly subtracting the divisor from the partial remainder. If the result becomes negative, it restores the previous value by adding the divisor back. These repeated restoration steps increase execution time.

Step 2: Restoring Division Workflow

For each bit:

1. Left shift (A, Q).
2. Subtract the divisor:
 - $A = A - M$
3. Check the sign of A:
 - If $A \geq 0$:
 - Set $Q_0 = 1$
 - If $A < 0$:
 - Restore: $A = A + M$
 - Set $Q_0 = 0$
4. Repeat for all bits.

Step 3: Performance Bottleneck

The inefficiency occurs when:

- After subtraction, the remainder becomes negative.
- The algorithm immediately adds the divisor back (restoration).
- This extra addition is repeated many times, increasing execution time.

Example:

$A = A - M$
If $A < 0$
 $A = A + M \leftarrow$ Restoration (extra operation)

These repeated restoration operations slow down the embedded system.

Step 4: Optimized Solution – Non-Restoring Division

The Non-Restoring Division Algorithm avoids immediate restoration.

Algorithm:

- If the previous remainder is positive, subtract the divisor.
- If the previous remainder is negative, add the divisor.

- Continue without restoring after every negative result.
- Perform one final correction at the end if the remainder is negative.

Step 5: How Non-Restoring Division Improves Performance

Restoring Division	Non-Restoring Division
Restores after every negative remainder	No immediate restoration
More addition operations	Fewer arithmetic operations
Higher execution time	Faster execution
Less efficient	More efficient for embedded systems

Final Answer

- Problem: Repeated restoration (adding the divisor back) after negative remainders causes unnecessary operations and increases execution time.
- Cause: Every negative subtraction requires an extra restoration step.
- Optimized Solution: Replace the Restoring Division Algorithm with the Non-Restoring Division Algorithm, which skips immediate restoration and performs only a final correction if needed.
- Benefit: Fewer arithmetic operations, reduced execution time, improved processor efficiency, and better performance in real-time embedded systems.

5. Debugging IEEE 754 Single Precision Floating-Point Addition

Step 1: Problem Analysis

A scientific application adds a very small number to a very large number using IEEE 754 single precision (32-bit) and produces inaccurate results.

Example:

- Large number = 1,000,000.0
- Small number = 0.000001

The small number may be lost because of limited precision.

Step 2: IEEE 754 Single Precision Format

- 1 bit – Sign
- 8 bits – Exponent
- 23 bits – Mantissa (Fraction)

Step 3: Debugging the Floating-Point Addition

1. Exponent Alignment

Before addition, the exponents must be equal.

- The number with the smaller exponent is shifted right.
- If the exponent difference is very large, many significant bits of the smaller number are discarded.

Issue: The small number loses precision during alignment.

2. Normalization

After addition:

- The result is normalized so that the mantissa has the correct format.
- The exponent is adjusted accordingly.

Issue: Normalization may remove additional low-order bits, reducing accuracy.

3. Rounding Error

Since only 23 mantissa bits are stored:

- Extra bits are rounded.
- Small values may disappear completely after rounding.

Issue: The final result may be the same as the large number, ignoring the small value.

Step 4: Source of Error

The inaccuracies occur because of:

- Large exponent difference
- Loss of significant digits during exponent alignment
- Normalization effects
- Rounding due to limited mantissa precision

Step 5: Optimization Techniques

1. Use Double Precision (64-bit)

- 11-bit exponent
- 52-bit mantissa
- Stores more significant digits and reduces rounding errors.

2. Algorithmic Adjustments

- Add numbers of similar magnitude first.
- Use techniques such as Kahan Summation Algorithm to reduce accumulated rounding errors.
- Avoid unnecessary floating-point operations that increase precision loss.

Final Answer

- Problem: Inaccurate results occur when adding very small and very large numbers using IEEE 754 single precision.
- Cause: Exponent alignment shifts out significant bits, followed by normalization and rounding errors.
- Optimization: Use double precision (64-bit) for higher accuracy and apply algorithmic improvements such as adding similar-sized numbers first or using Kahan Summation to minimize rounding errors.
- Result: These techniques significantly improve numerical accuracy in scientific applications.